



vMES: Virtualization aware manufacturing execution system



Octavian Morariu^{*}, Theodor Borangiu, Silviu Raileanu

Centre for Research & Training in Computer Integrated Manufacturing and Robotics, University "Politehnica" of Bucharest, Bucharest, Romania

ARTICLE INFO

Article history:

Received 9 February 2014

Received in revised form 26 July 2014

Accepted 18 November 2014

Available online 25 December 2014

Keywords:

MES

Virtualization

Private cloud

Shop floor

Resource allocation

ISA-95

ABSTRACT

The large scale emergence in the last decade of various cloud solutions, ranging from software-as-a-service (SaaS) based solutions for business process management and implementation to very sophisticated private cloud solutions capable of high performance computing (HPC) and efficient virtualization, constitute the building blocks for engineering the next generation of flexible enterprise systems that can respond with great agility to changes in their environment. These new technologies are adopted at a certain level by manufacturing enterprises in order to advance in a new era of mass customization where flexibility, scalability and agility are the differentiating factors. In this context, this paper introduces the virtualized manufacturing execution system (vMES), an intermediate layer in the manufacturing stack, and discusses the advantages and limitations offered by this approach for manufacturing enterprises. A classification of MES workloads based on the ISA-95 function model is presented, focusing on the virtualization techniques suitable for each workload, considering the algorithms and technologies used and the virtualization overhead. A pilot vMES implementation using a parallel process for smart resource provisioning and automatic scaling is also presented. The pilot implementation using six Adept robots and one IBM CloudBurst 2.1 private cloud and an ISA-95 based MES is described; the virtualization sequence is analyzed in several scenarios of resource workload collocation on physical cloud blades with and without perturbations.

© 2014 Elsevier B.V. All rights reserved.

1. Introduction

The trend towards virtualization of software systems gains more and more traction in modern enterprises in this decade. There are a series of immediate benefits offered by virtualization for almost any kind of software system; better resource utilization, improved flexibility and higher availability, which translate directly in reduced operational costs for the enterprise. Virtualization also represents the main enabling technology for cloud computing, regardless of the delivery method considered. Today enterprises can take advantage of public cloud offerings to provision virtual workloads (IaaS), applications on specific platforms (PaaS) or access in multi-tenant applications (SaaS) in a pay-per-use model, with no initial investment required. At the same time, complex software solutions can be based on various combinations of cloud domains and delivery methods. These advances in how technology can be provisioned and consumed across the Internet in real time open the door to the development

of new business models and give a new degree of agility to the IT departments supporting core business functions of any enterprise.

From a location domain perspective we can consider four established cloud delivery models: public clouds, community clouds, private clouds and hybrid clouds. *Public clouds* are designed to be used by a variety of different customers (users or enterprises) and are usually built around open standards. From a cost perspective the services can be either free or in a pay per usage model. *Community clouds* are shared by several organizations and support a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations). It may be managed by the organizations or a third party and may exist on premise or off premise. *Private clouds* are built inside enterprise boundaries and are designed to offer services to business units of that enterprise only. Private clouds are cost effective only for large enterprises that can assure an optimal usage of cloud resources. For small enterprises a private cloud would be an overhead from a cost perspective, as they will still have to own and maintain the hardware and software infrastructure. *Hybrid clouds* are a combination of the above mentioned two models, where both public and private services are composed in order to provide enhanced functionality. Hybrid clouds involve facilitating the data sharing between private and public domains.

^{*} Corresponding author. Tel.: +40 741330003.

E-mail addresses: octavian.morariu@cimr.pub.ro (O. Morariu), theodor.borangiu@cimr.pub.ro (T. Borangiu), silviu.raileanu@cimr.pub.ro (S. Raileanu).

Manufacturing enterprises have started to adopt cloud computing, especially in the upper side of the software stack; enterprise resource planning (ERP), supply chain management (SCM) and customer resource management (CRM) areas. One of the established examples in these regards is Salesforce.com [1] which offers a complete range of business supporting applications in a SaaS delivery model. However, cloud adoption in industry is driven mainly by cost reduction and so it is opportunistic in nature. There is no systematic approach to cloud adoption that can align the manufacturing industry organizational standards like ISA-95 to the above mentioned cloud delivery models, in a consistent fashion. ISA-95 [2] defines 5 levels for the hierarchical organization of manufacturing enterprises, as follows:

- Levels 0, 1 and 2 represent the *process control levels* and their objective is to control the physical shop floor equipment in order to execute the actual production operations that result in one or more finished products;
- Level 3 is the MES (*Manufacturing Execution System*) level and consists of several activities that have to be executed in order to prepare, monitor and complete the production process executed at level 0, 1 and 2. These activities are scheduling, quality management, maintenance, production tracking, and others depending on the specifics of each enterprise. Several formal specifications for MES have been attempted [3]. Private clouds, in IaaS delivery method are suited for this ISA level.
- Level 4 is the ERP (*Enterprise Resource Planning*) level, where financial and logistic activities are executed. These activities are not integrated in real time to the production and include: long term planning, marketing and sales, procurement, and so on. Public clouds, in SaaS delivery method are suited for this ISA level.

The term *cloud computing* is widely used to describe any information system that is delivering computing services over the Internet [4]. From this perspective, cloud computing represents a fairly new model for delivering IT services to consumers (end users, enterprises, organizations, etc.) by providing computation power, software, data access, networking and storage in an *on demand* fashion. The main novelty in this delivery model is that cloud providers are offering computing services in the form of a utility, similar to the concept used to deliver electricity to provide energy to end users in the past. The concept of cloud computing was first introduced back in 1960s by John McCarthy [5]; however, due to technological limitations of that time, his vision never got enough traction in the industry to materialize. Since then, there were several technological breakthroughs in the virtualization area that allowed the emergence of cloud computing.

A common used definition of cloud computing is the one provided by National Institute of Standards and Technology [6], which defines the main cloud characteristics, as follows:

- *On-demand self-service*. A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service's provider.
- *Broad network access*. Capabilities are available over the network, being accessed through standard mechanisms that promote usage by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and PDAs).
- *Resource pooling*. The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a sense of location independence in that the customer generally has no control or knowledge over the exact location of the provided resources but may be able to specify a location at a higher level of

abstraction (e.g., country, state, or datacentre). Examples of resources include storage, processing, memory, network bandwidth, and virtual machines.

- *Rapid elasticity*. Capabilities can be rapidly and elastically provisioned, in some cases automatically, to quickly scale out and rapidly released to quickly scale in. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.
- *Measured Service*. Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported providing transparency for both the provider and consumer of the utilized service.

Additionally, other cloud characteristics can be considered, as follows:

- *Multi-tenancy* refers to the fact that the same resource instance is being used concurrently by multiple cloud customers (end users or organizations). This can be seen at multiple layers, because in each model there is some resource being shared: the actual hardware box, the hypervisor, the platform or the actual software application. This approach brings a huge advantage in terms of resource usage optimization, but raises a number of challenges, specially related to customer data security.
- *Pay-per-use* model refers to the charge back model adopted by cloud providers. This allows a low start-up investment from customers and at the same time allowing them to use high end technologies.

The most important adoption driver for cloud computing is the conversion of traditional computing systems to virtualized environments. The virtualization of a computing system in general means that hardware and software resources are organized and managed in a pool model, allowing multiple beneficiaries to use these resources as needed. This approach leads directly to a higher resource utilization, which represents the primary goal and benefit for any form of cloud computing. Historically, computer systems like databases, Business Intelligence (BI) servers and so on, were deployed on dedicated physical infrastructures, or in other words, dedicated servers and storage usually interconnected with dedicated network as well. Some of the most common reasons for this practice were the poor standard adherence of legacy implementation, very narrow certification margins imposed by software vendors, no real option to impose hard resource utilization quotas at operating system level, and many others. The result of this practice was that the overall resource utilization was below 20% on average [7]. The pressure to reduce costs by optimization of resource utilization has set the scene for a fundamental conceptual change towards virtualization as long term strategy inside large enterprises.

Virtualization technologies available today allow creating virtual instances of various physical devices of a computing environment like CPUs, memory, storage, networking, USB ports and various other devices and ports as required. The virtualization of these components is achieved by introducing an intermediate software layer between the physical resources and the operating system layer called *hypervisor* [8–10]. A hypervisor is transparent for the guest operating system and manages the virtual resources and their mapping to the physical resource layer. In the last decade, hypervisors have evolved and provide a rich set of features allowing not only fine grained management of physical resources but also many real time optimizations. The scenario where private clouds are used in the industry is where the enterprise strategic direction is to manage the information storage and processing

inside the enterprise premises, but at the same time to take advantage of virtualization and thus improve resource utilization. Some of the direct advantages of moving from legacy computer systems to private cloud architectures are: reduction of power consumption, data centre floor space, cooling requirements and others. Technically, private clouds can be built from existing computing resources by using a cloud management software stack or can be purchased, as turn-key solutions, from established cloud vendors like IBM, Oracle, Microsoft, RedHat and others. These commercial solutions typically come as a set of blade servers, specialized storage devices, network devices and a cloud management software stack that implements the workload discovery functionality, service catalogue, provisioning workflows and basic administration of virtual resources. At the same time, private clouds respond implicitly to security challenges associated with other delivery methods [11–13].

For manufacturing enterprises these cloud offerings, ranging from software-as-a-service (SaaS) based solutions for business management functions, to very sophisticated private cloud solutions offer the building blocks required for constructing flexible enterprise systems that can respond to variable customer demand and environmental changes with great agility [14]. Manufacturing enterprises have to adopt these new technologies in order to advance to a new level of mass customization where flexibility, scalability and agility are the differentiating factors for real-time control manufacturing systems. The end goal would be to enable a *customer-driven resource allocation at MES level* through real time analysis of the computing capacity required to manufacture the products associated with a given product batch.

This paper aims at providing a structured approach to cloud adoption and MES virtualization for manufacturing companies that are organized using the ISA-95 models, focusing especially on level 3, where private clouds can offer support for a virtualized MES. The paper presents the main workloads involved in the virtualization process, the triggers used to assure scalability, efficiency and flexibility to the solution and a *virtualized layer for MES* (vMES) system designed to take full advantage of the virtualized platform where it is hosted. The research challenge here consists in the design of a system capable to drive the vMES solution, adapted to the ISA-95 workload model. The solution provided in this paper is based on a SOA workflow implementation with both synchronous and asynchronous execution, capable of controlling the manufacturing process and optimal resource allocation for cloud workloads.

2. Related work

Research done by Wang and Xun [15] introduces Interoperable Cloud-based Manufacturing System (ICMS). ICMS provides a Cloud-based environment integration of existing and future manufacturing resources by packaging them with a Virtual Function Block mechanism. This mechanism promotes and utilizes standardized descriptions of resources and processes. At the same time, resource sharing in a cloud computing environment was explored by Lei [16] focusing on enabling cooperative work between enterprises to achieve global manufacturing capabilities. A comprehensive study on the utility model and utility equilibrium of resource service transaction in cloud manufacturing is provided by Cheng et al. [17].

Cloud manufacturing (CMfg) was introduced [18] as a networked and service-oriented manufacturing model, focusing on the new opportunities in networked manufacturing (NM) area, as enabled by the emergence of cloud computing platforms. The cloud based service delivery model for the manufacturing industry, like product design, batch planning, product scheduling, real time manufacturing control, testing, management, and all other stages

of a product life cycle are described by Xu [19]. Similar concepts were discussed by Zhang et al. [20], with some advanced focus on typical characteristics of CMfg in regards to the key technologies in implementing a CMfg service platform.

Another important research area is represented by the study of resource virtualization techniques and resource sharing in manufacturing environments. Shi et al. [21] proposed an innovative model for resource sharing in grid manufacturing. This framework is composed of several layers: a network infrastructure layer, a manufacturing resource aggregation layer, a manufacturing resource management layer, a manufacturing service application layer and a portal layer. The manufacturing resource hierarchy model (MRHM), which consists of a manufacturing resource layer, a resource expressing layer and a resource interface layer, is proposed and discussed. In the same research area [22] and [23] argue that resources and resource capabilities virtualization and modelling are representing the starting point for manufacturing services encapsulation in the cloud. This research group provides a virtual description model of manufacturing resources that includes both functional and non-functional features of manufacturing resources. Additional research done by Luo et al. [24] expands this concept and provides a modelling and description method of information for multifunctional manufacturing capability in a CMfg system.

The CMfg and resource virtualization concepts presented in the above discussed articles are focusing on resolving the inter-enterprise cooperation problem in order to achieve optimized manufacturing using the emerging cloud technologies as enabler technology. While there is a solid understanding in the research community on the fact that the CMfg concepts are correct and represent the future of manufacturing, we argue that MES virtualization concepts described in this paper are prerequisites for effective adoption of cloud technologies and especially virtualization at MES layer. The concepts introduced in this paper are meant to fill the gap between the theoretical concepts behind CMfg and the current MES related challenges in the context of virtualization adoption for manufacturing enterprises.

3. MES virtualization concepts

A cross section through the IT landscape of an ISA-95 compliant manufacturing enterprise would show typically a three layered architecture as illustrated in Fig. 1. The cloud adoption strategy must result in an architecture that is robust enough to assure the information flow is in sync with the material flow at all times [25], and at the same time, flexible enough to allow dynamic reconfiguration and SOA governance. Some examples in these area are holonic based MES systems as described in literature [26,27]. Standards alignment is also an important factor to consider (ebXML, 2013; STEP, 2013; OAG BOD, 2013). One commercial example in this direction is the IBM Manufacturing Integration Framework [28], implemented successfully in automotive and electronics industries by several large organizations. Similar SOA-based solutions, targeted for manufacturing enterprises, are available from other software vendors and integrators.

The lower layer consists in the so-called shop floor or physical layer and maps to ISA-95 levels 2, 1 and 0. This layer contains the resources, transportation devices and “intelligent products” involved in the manufacturing process. The physical software workloads at shop-floor layer are most of the times proprietary (custom hardware and custom software) or in some cases semi-proprietary (standard hardware and operating system and custom software) and are responsible with direct real time control of the physical resources and of the material flow on the shop floor [29]. Tightly coupled with the physical layer is the Manufacturing Execution System (MES), which maps with level 3 of ISA-95

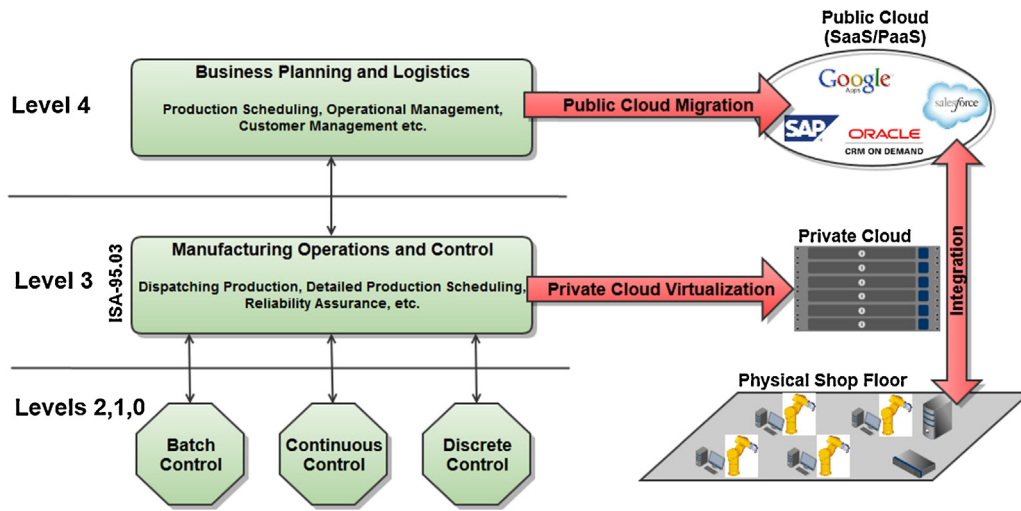


Fig. 1. From service oriented manufacturing systems to cloud.

specification. The communication pattern between MES and the physical workloads in shop floor is, from a SOA perspective, *service choreography* with point to point message exchange in real time. When considering migration to cloud environments, these characteristics of MES specific workloads can be assured by private cloud implementation with *workload virtualization*. In contrast, public clouds, even in a IaaS delivery model are not able to provide a comparable response time, mainly due to network latency. Private clouds have the advantage that are located within the enterprise premises and so network latency can be kept very low.

In this context, MES virtualization involves migration of all MES workloads that were traditionally executed on physical machines to the data centre, specifically to the private cloud infrastructure as virtual workloads. The idea is to run all the control software in a virtualized environment and keep only the physical resources with their dedicated real time controllers on the shop floor. This separation between hardware resources and software that controls them provides a new level of flexibility and agility to the manufacturing solution. Depending on the manufacturing system design and implementation, there can be several types of workloads that can be identified. The following section contains a classification of these workloads done from a virtualization perspective.

3.1. ISA-95 Level 3 functions and workloads

The most important functions defined by ISA-95.03 at MES level (Level 3) are presented in Table 1, together with their corresponding descriptions:

Table 1
ISA-95 Level 3 functions.

ISA-95 Level 3 function	Description
Detailed production scheduling	Deals with the algorithms and computation required to calculate the operation sequence for each product in the batch and the shop floor resource allocation for each operation.
Production dispatching	Refers to the process of dispatching the detailed production schedule after it is computed to the shop floor resources, intelligent products, conveyors and human workers.
Production execution	Drives the actual execution of the detailed production schedule on lower level systems.
Production tracking	Tracks each product during the execution phases from the initial entry in the manufacturing process until completion.
Resource management	Deals with the management of the shop floor resources in terms of availability, utilization, raw materials stocks, and so on.
Historical data management	Assures traceability of all operations performed on every product. The complexity involved with this function depends largely on the nature of the products being manufactured and on the legal requirements that apply.
Quality assurance	Assures the product meets the acceptable quality criteria. This can be done by video inspection, X-ray scanning or by other specific means.
Manual operations	Assures the capability of the MES system to interact with human operators in various scenarios.
Automated control	Provides capabilities for automatic re-configuration of the system, allowing the system to respond to unexpected events, like resource breakdown or rush orders.

The relationship between these functions is illustrated in Fig. 2, where the red arrows show the explicit communication paths between the modules using the Manufacturing Service Bus concept. A more detailed view of the MSB integration in the context of cloud adoption is presented in section 3.2 of this paper.

In most implementations the above defined functions are mapped in separate workloads at MES layer. These workloads have a set of special characteristics, as required by the MES function they implement. The remainder of this section presents an analysis of these workloads from a virtualization perspective, focusing on the workload profile and on scalability requirements in regards to system load.

The *detailed production scheduling* is the process that determines the operation sequencing and the resource allocation for the entire product batch. The scheduling problem is often a NP-complete problem and finding a near optimal solution can take a significant amount of computation time. The workload is active before the actual fabrication begins and once the schedule is computed, the workload becomes idle. The workload profile tends to be a CPU-intensive profile and is used only in the initial phase. The scalability of the workload depends highly on the algorithm used for schedule computation. An algorithm that can be executed in parallel would allow for horizontal scaling of the workload (i.e., provisioning of multiple parallel virtual machines for parallel computation depending on the problem/batch size). On the other hand, a monolithic algorithm would require scaling by resource allocation to a single virtual machine (i.e., by allocating more CPU cores or physical memory to a given virtual machine executing the algorithm).

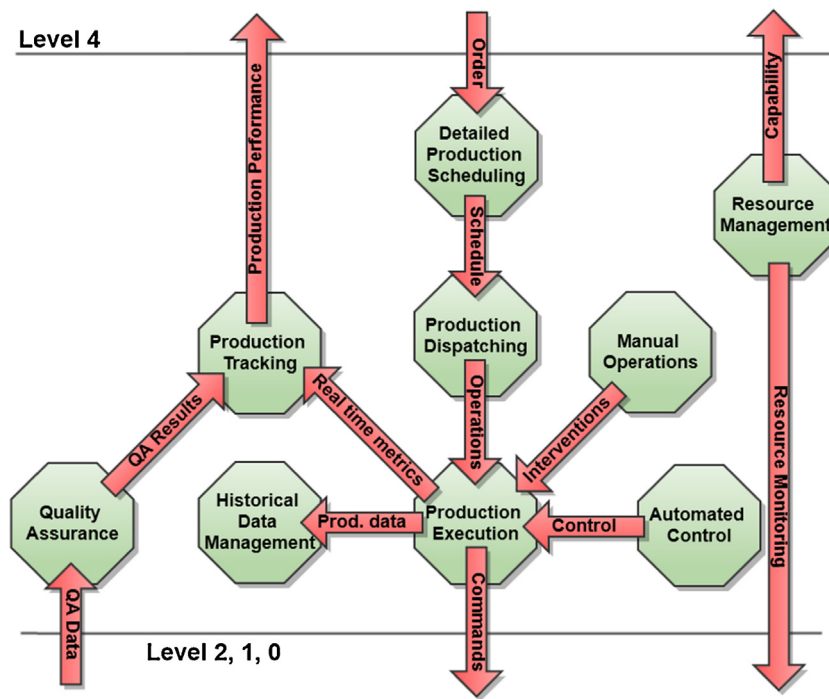


Fig. 2. Level 3 modules and interconnections.

The *production dispatching workload* has similar temporal characteristics as detailed production scheduling workload, in the sense that it is active for a short duration after the schedule is computed and before execution starts. For this reason, in most implementations this workload is collocated with the scheduler workload. The schedule dispatching operation itself is not resource intensive, especially in a MSB integrated environment, as it is just responsible for sending a set of messages across the MSB.

The *production execution workload* deals with the actual hierarchical execution of the schedule by generating real time commands to the lower levels of the stack, specifically to the shop floor devices and controllers. The production execution workload becomes active after the detailed schedule is dispatched and remains active throughout the entire manufacturing process. As illustrated in Fig. 2, the process execution workload interacts in real time with many other workloads and with the shop floor components, in an event driven architecture. The workload has a high I/O profile, as data flows from various directions via events from other MES components or from shop floor devices. The production execution workload can be scaled horizontally using a load balancer mechanism. Closely related to the production execution workload is the *automated control* workload. The automated control workload handles the unexpected scenarios that affect the initial computed schedule (i.e., unexpected resource breakdown, stock depletion, rush orders). The implementation of this function involves a meta-heuristic class algorithm that can find the best possible solution in an un-expected scenario, making this workload a resource intensive one. However, the workload is not active in normal operation of the system, where the detailed production schedule can be followed by the system. Essentially, this workload would be provisioned only if certain events are occurring.

The *manual operations workload* consists usually in a Web based application where human operators can handle manufacturing events (i.e., some implementations use the concept of *service tickets*, or *service requests* that can be assigned to certain engineers and used as tracking items for the work done). Typically, this workload is active throughout the entire manufacturing process, as events that need human intervention can occur at any time.

From a virtualization perspective, the workload is not resource intensive and can be collocated with other workloads.

The *resource management* function is implemented as a simple resource monitor that periodically checks the status of the manufacturing resources, consolidates this information, and based on it computes the available capacity of the system. This capacity is considered by the upper layer (level 4) when deciding the manufacturing schedule. The workload is not resource intensive and can be collocated with the manual operations workload.

The *historical data management* function consists usually in a relational database management system (RDBMS) that stores the manufacturing data in a structured and accessible format. Depending on the granularity of the data stored and the size, the workload would need to access a large disc space and is would have an I/O intensive profile.

The implementation of the *quality assurance* workload depends largely on the QA process implemented for the product being manufactured. It may consist in a simple implementation that compares some of the acquired product features (dimensions, shape) against some predefined thresholds, or can become quite complex when using video defect detection techniques or other advanced scanning mechanisms and image processing algorithms. In a virtualized environment, the resources allocated to this workload would vary based on the product type and the QA process defined for it. The *production tracking* workload keeps track of the manufacturing execution process and on the QA data, and reports the progress across the MSB to the upper level. The workload essentially deals with simple data aggregation, with no complex computation being involved, so it is not a resource intensive workload. The workload remains active throughout the entire manufacturing process.

The above MES workloads are interconnected using the *manufacturing service bus* (MSB), which is discussed in detail in the following section.

3.2. Manufacturing service bus characteristics

The MSB characteristics for MES in a physical (non-cloud) implementation are presented in some of our previous work

[30]. In this section we present the adaptation of MSB concept in the context of a private cloud implementation. At the heart of the service oriented manufacturing system is the enterprise service bus/manufacturing service bus coupling. The Enterprise Service Bus (ESB) was first proposed by [31] as a software architecture having a set of key characteristics:

- Message routing and control across enterprise components;
- Decoupling of various modules by asynchronous messaging, replacing point to point communication with the common bus architecture;
- Promote reusability of utility services, reducing the number of redundant services across the enterprise;
- Provide transformation and translation of messages to allow easy integration of legacy applications;
- Provide an engine for workflow execution.

As present, all major software companies provide solid commercial implementation of ESB on top of their SOA offerings: IBM WebSphere ESB, Microsoft BizTalk Server or Oracle Enterprise Service Bus. Along with commercial implementation there are also open source solutions: JBoss ESB, Open ESB, Apache ServiceMix and others.

The Manufacturing Service Bus (MSB) integration model is an adaptation of ESB for manufacturing enterprises. The MSB has more specific characteristics in addition to the ones inherited from the ESB:

- *Event driven communication*: at shop floor level, during the manufacturing process there is a high number of events generated that need to be handled by specific components. For example, when a pallet arrives in a given position on the conveyor belt, a sensor detects the associated RFID tag and generates an event. This event needs to be dispatched to the relevant resources in order to be processed by the scheduling module or by the actual robot that performs an operation. The main role of the MSB implementation is to perform the event dispatch operation allowing shop floor components to exchange information in an event driven fashion.
- *Workflows*: along with event dispatching, the MSB has the ability to launch and execute predefined workflows associated with specific events. Workflows consist in a set of successive operations, either automated or manual (human interventions). Workflows are typically required for exceptional events that require complex logic to handle them, like unexpected resource breakdowns or rush orders. Another advantage of grouping complex logic in workflows is that the workflows are “external” in reference to the MSB, from the implementation point of view.
- *Message transformation*: the shop floor level integrates a wide range of modules, from software schedulers to various hardware devices (robots, sensors, etc.). From a communication perspective, the protocols and the message formats used can be a simple +5 V DC signal, proprietary line protocols or even high level TCP based protocols. The MSB role is to transform these messages to and from these proprietary protocols in a common standardized format. This is done by allowing the development of message convertors at any entry point and exit point of the bus.
- *Synchronous and asynchronous communication*: the MSB implementation offers both synchronous and asynchronous communication modes. The synchronous mode causes the sender of the message to block until the response is received and so is implicitly bidirectional. The asynchronous mode is using a queue based mechanism, where the sender submits the message and from where the receiver picks it up at a later time. This allows decoupling the execution of the sender from the one of the receiver. At the shop floor level both communication modes are useful.

- *Message persistence*: when asynchronous mode is used, the messages reside in logical queues from where they are consumed. The MSB implementation stores the queues in a persistent highly available storage that allows production state recovery in case of a system crash. The MSB can use a network file system or a distributed database as a repository for the message queues.

In a cloud-based implementation, the MSB layer with the above mentioned characteristics becomes a distinct component called *Cloud Integration Mediator*, bridging the gap between the local private cloud implementation and the public cloud services used in a SaaS fashion, represented by level 3 in ISA-95 specification. The mediator is conceptually built on top of SOA and must have ESB and B2B (business to business) capabilities to integrate with applications provided via Internet. Implementing the mediator module is fundamentally an *integration effort* that spans across all levels in the enterprise. This MSB mediator bridges the workloads and functionality used from public clouds (in SaaS model) with the lower layers of ISA-95 implementation.

The information flow in a manufacturing enterprise is governed by business applications handling customer orders, supply chain operations, business intelligence and so on. These applications are traditionally operated and maintained by the company's own IT department using dedicated hardware stacks. This approach requires huge up front investments to setup the systems, high operational costs and most of times deliver below the intrinsic capabilities of the software products [32]. The large scale availability of applications delivered on a software-as-a-service (SaaS) model raises an important opportunity for manufacturing enterprises to *migrate* from in-house implementations of complex business applications to public cloud solutions available either in multi-tenant mode or dedicated hosted solutions.

4. vMES virtualization details

The virtualized MES architecture proposed in this paper is driven by a central BPEL process that orchestrates the workload provisioning processes and the dynamic scaling of the solution according to dynamic events generated during the manufacturing process.

The vMES process described here is designed to integrate with virtually any MES implementation that follows the functions and structures defined by ISA-95 level 3. The vMES process fills the gap between the virtualized workloads and the underlying private cloud architecture, allowing intelligent and efficient usage of cloud resources for MES execution. The BPEL process proposed is illustrated in Fig. 3.

The BPEL process is invoked by the Customer Order Management module (situated at Level 4 in the enterprise hierarchy) [33], when a customer order needs to be executed. The customer order encapsulates the production schedule of the entire product batch ordered by the customer, together with the time constraints associated. The BPEL process itself is exposed as a web service end point (vMES_EndPoint) and can be invoked through the upper level ESB implementation in a standard fashion. The process is designed in two sections (Workloads_Management and Event_Management) that execute in parallel.

Section I: Workloads_Management starts with a Java embedding activity that provides an algorithm that computes the initial provisioning plan, based on the product batch. Specifically the product types and the batch size are the two components that would decide what workloads would be provisioned at MES level and what kind of resources would be allocated to these workloads. This computation is based on predefined rules for variable workloads (if product is of type X, then QA is done with

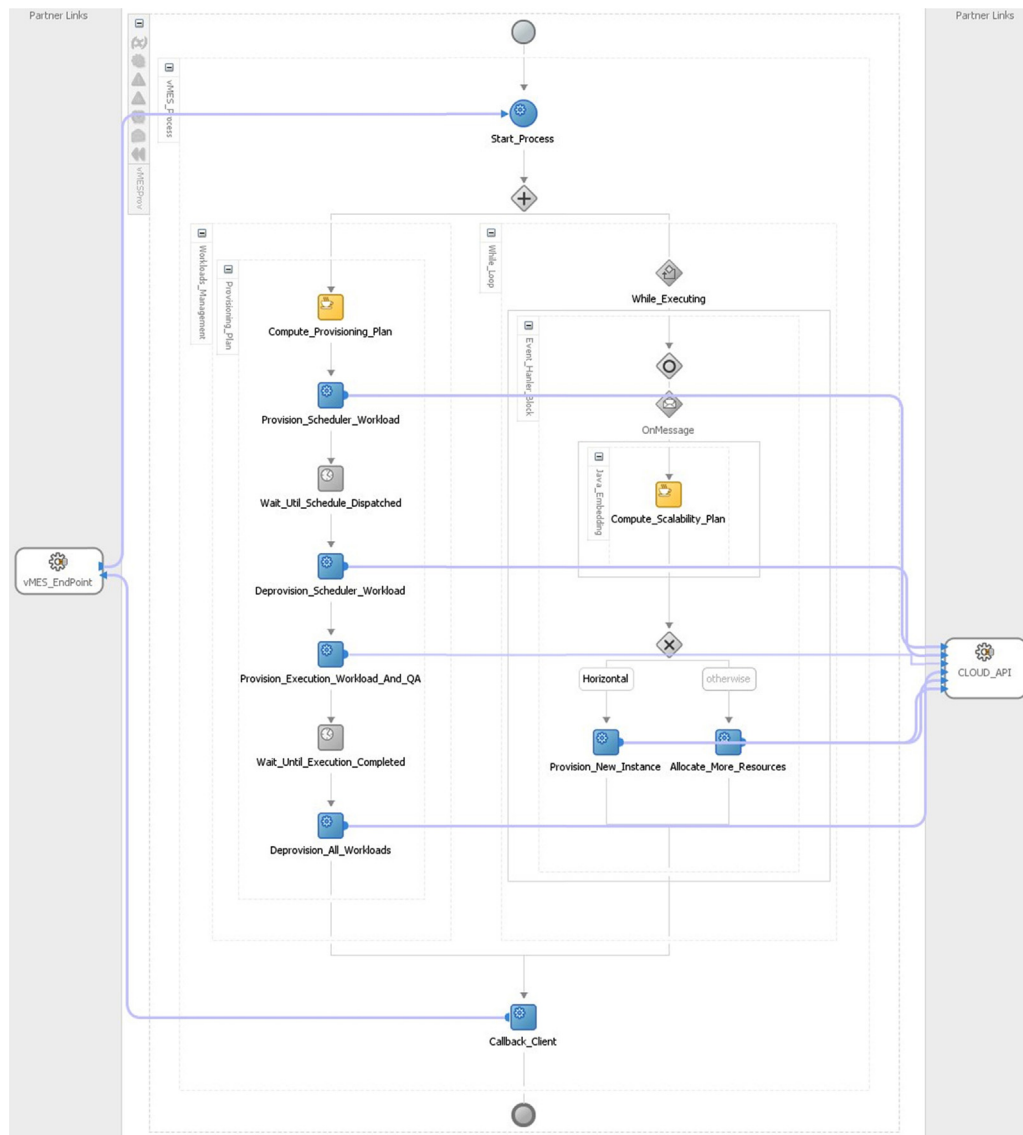


Fig. 3. vMES BPEL process design.

workload W) and predefined thresholds for fixed workloads (if product batch size >10 , then allocate 1 additional core to the scheduler workload). The next activity provisions the *detailed production scheduling* workload. This high specs workload computes the operation sequencing and resource allocation for the whole product batch horizon. As this computation can take a long time for large product batches with many operations per product, the BPEL process waits asynchronously until the entire schedule is computed and dispatched via the MSB. At this point the *detailed production scheduling* workload is de-provisioned, as it is not required anymore for the product batch execution. The resources associated to this workload are released to the pool. The next step is represented by the provisioning of the *production execution* and *quality assurance* workloads. The process then waits for the execution to complete, and de-provisions the remaining workloads, releasing the resources to the private cloud resource pool.

Section II: Event_Management consists of a *while loop* that waits for generated events. Each event is handled by the *Compute_Scalability_Plan* activity. This activity determines the scalability action that would best respond to a real time monitoring event, according to the workload scalability profiles. For example, if

the scheduler workload generates a *High_CPU* event (i.e., indicating that the CPU consumption is above a pre-configured threshold) a second node with the workload is provisioned as a response. The other alternative is to allocate more resources for a given instance. For example, if the *historical data management* workload triggers a *Low_Disk_Space* event, a new volume would be allocated to the workload as a result. Both types of scaling are supported by the implementation of the process, and by most private cloud implementations through the hypervisor features available. The events are handled throughout the entire manufacturing process.

When the execution is finished, the vMES BPEL process performs a callback operation on the level 4 Customer Order Management module, announcing the execution finalization.

5. Pilot implementation and experimental results

The virtual MES process introduced in this paper was evaluated in the context of a pilot manufacturing system of the CIMR Centre within University Politehnica of Bucharest, presented in Fig. 4. The goal of this pilot implementation is to virtualize the existing MES

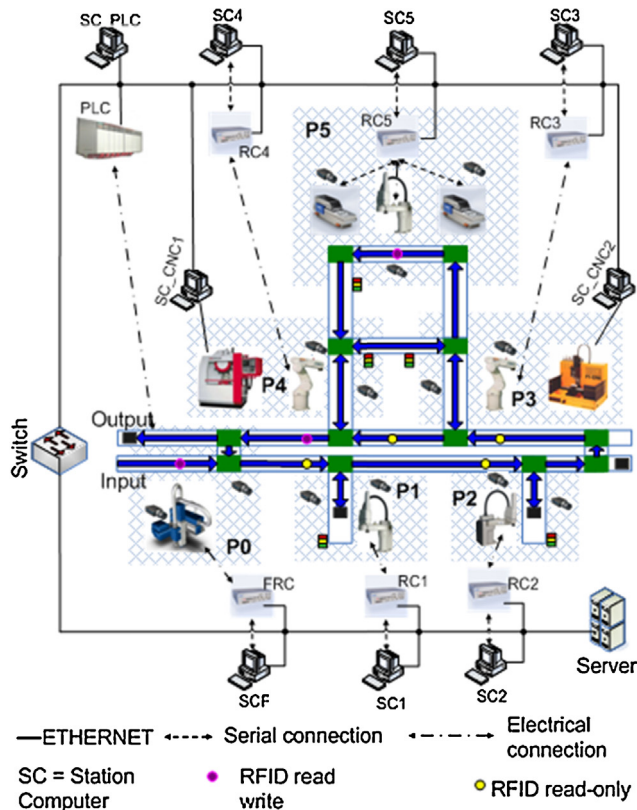


Fig. 4. Manufacturing system (CIMR Centre): layout and partial view of equipment.

implementation and analyze the impact caused on the resource utilization and scalability capabilities, considering that the MES functionality itself does not change in the virtualized environment. The vMES triggers relay on a monitoring solution that monitors the workload operating parameters (i.e., CPU usage, memory usage, disc usage, I/O operations per second, network traffic, and many others).

The private cloud infrastructure is provided by an IBM CloudBurst 2.1 medium size instance (Fig. 5). The IBM Cloud Burst system is an offering based on IBM Blade Center that adds virtualization capabilities using the VMware ESX 4.1 hypervisor, VMware VCenter 4.1 and enhanced administration capability by leveraging the Tivoli Service Management Stack. The virtualization platform is powered by 14 blade servers with two Intel processors with 6 cores each at 2.8 GHz and 12 Mb L3 cache. The installed

memory is 72 GB for each blade. The estimated capacity is in the area of 400 concurrent virtual machines, considering an optimum scheduling with mixed CPU/I/O profiles. For this pilot implementation, only 4 blades were used for MES virtualization, isolated from the rest of the load. This was required in order to assure accurate measurement of message propagation between shop floor devices and MES workloads.

The virtualization process was performed in several steps. The first step consisted in the virtualization of the MSB component that represents the communication platform for all MES workloads. The strategy is to use two virtual network adapters, one dedicated for management operations, specifically the integration with the ISA-95 level 4 systems, and the other for direct Ethernet connection with the other MES workloads. The virtual machine was allocated with two virtual cores and 2 GB RAM.

The second step involved the virtualization of the detailed production scheduler and the automatic control module, residing on a physical xSeries 3500 Server. Due to the system design the decision was taken to split the workload in two virtual machines, one dealing with the detailed production scheduling in hierarchical mode and one with the heterarchical operating module (automatic control) implemented with multi agent system inside the MSB architecture. This approach decouples the two systems that used to share the same physical server and so improves the overall reliability of the system. The third step involved the virtualization of the static workloads: production tracking, human intervention and quality assurance workloads for several product types. The virtualization process resulted in 11 individual virtual machine templates, published in the service catalogue of the IBM CloudBurst private cloud; one for each ISA-95 function and one additional for the MSB implementation.

A typical workload provisioning across four individual blades using the vMES process is illustrated in Fig. 6. The workloads are represented in different colours and the provisioning and de-provisioning times are also shown. In the pilot implementation, due to the simplicity of the solution, the workloads have a similar provisioning and de-provisioning time of about 2 min. However, in real life implementations it is expected that the MES workloads would have a higher and variable provisioning time.

The scenario where a perturbation affects the system, preventing the initial schedule to be executed, is illustrated in Fig. 7. The vMES process responds by dynamically provisioning the Automatic Control workload, which will take control of the remainder of the batch execution, according to the specific MES implementation. This is also referred to in literature as heterarchical operational mode for MES execution.

The initial tests performed in the lab environment indicate that the virtualization at MES layer is a viable solution, as the response times for the provisioning, de-provisioning and scaling functions are much lower than the lifetime of the workloads. For example, the 2 min provisioning time is acceptable when considering that the production of the batch, while the workload is active, takes hours. So, even if an overhead is introduced by these operations compared to the physical systems, this overhead is acceptable for MES implementations. Is important to note that some workloads which are directly integrated (or control) physical systems require a very low network latency during execution, but the provisioning/deprovisioning times are not that tight. While others, like QA related workloads can tolerate a high network latency but require low provisioning/deprovisioning times.

6. Benefits and limitations of vMES approach

In the last decade, server virtualization benefits have been outlined and discussed by both research and industry to a great extent. While all server virtualization benefits apply implicitly to

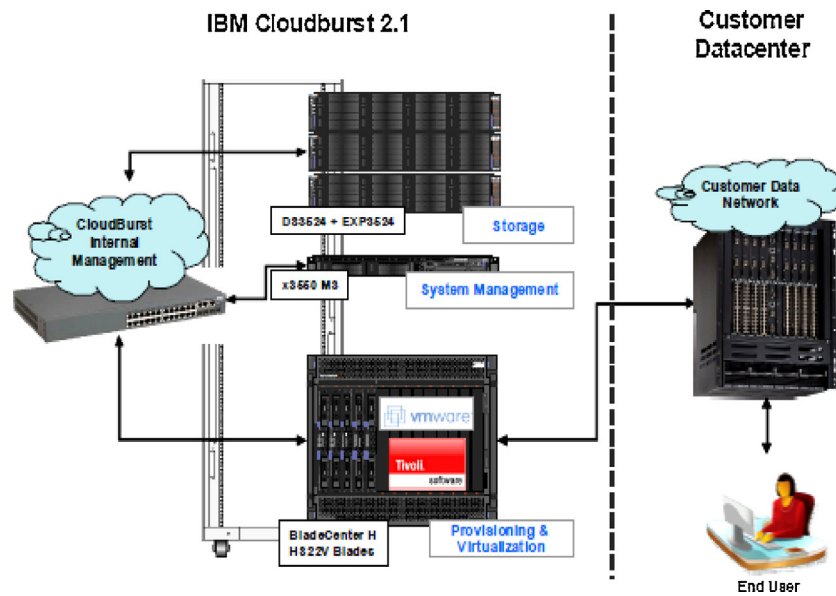


Fig. 5. IBM CloudBurst 2.1 private cloud solution.

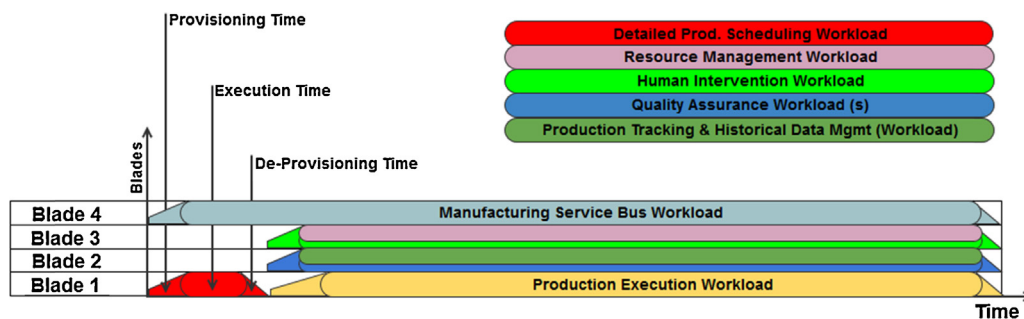


Fig. 6. Workload allocation in normal operation (no perturbations).

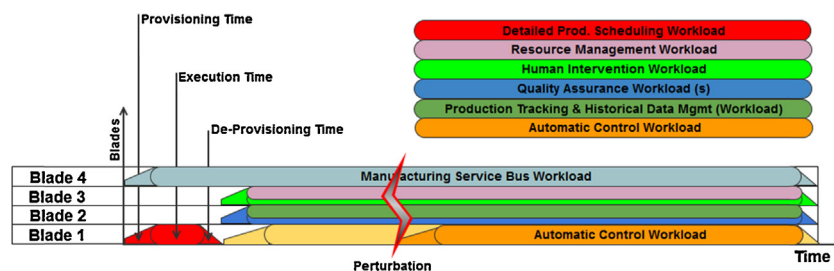


Fig. 7. Workload allocation in perturbed operation (resource breakdown scenario).

MES virtualization, there are some additional benefits that are specific for this industry in the context of MES virtualization.

6.1. Virtualization advantages

The principal advantages of MES virtualization are:

Decoupling: MES workload virtualization allows a separation between the physical resources and the software information system. The most important advantage introduced by decoupling is the possibility to have multiple versions of the software workload with different configurations and switch between them as needed. This capability particularly useful when the initial configuration or changes need to be tested and

implemented on the production line (e.g., resource team reconfiguring). At the same time, based on this, a rollback option is implicitly available.

Redundancy: with virtualized MES a redundancy mechanism with active/active or active/passive workloads can be implemented. In traditional implementations of manufacturing cells, implementing a MES redundancy requested the utilization of a separate physical server component, and so it had a high operational overhead. However, without redundancy it is impossible to assure high availability for MES components. In other words, if a specific server component at MES level faced a problem at runtime, the whole MES control would become unavailable. A private cloud implementation would be able to detect a workload failure and switch to the redundant

workload in a very short time, assuring a high uptime for the MES workloads.

Energy Efficiency: server virtualization is improving dramatically the energy footprint of the workloads. In most cases, the energy consumption is reduced to 15–20% of the physical implementation. In the pilot implementation described above, by virtualizing the physical MES servers, the estimated energy cost savings per year are \$27,000, according to the VMware Green Calculator (VMware, 2013). These estimates include also the cooling required for the physical workloads.

High Performance: some MES functions would require high performance computers for specific tasks. For example, real time image analysis for error detection (QA) or for material selection (recognition and locating) would require high CPU resources. With virtualization and private clouds, such resources can be allocated virtually according to the needs of the specific manufacturing line. A practical example is represented by the scenario where the manufacturing line is executing at 100 products per hour, with a requirement to process 1000 images per product. For this, 8 virtual processors can be allocated to the image analysis workload. If the manufacturing line will later execute at 50 products per hour, the processing power required will be half, so the workload can be configured with 4 virtual processors. This fine adjustment of the processing power to the actual product manufacturing requirements is not possible without virtualization.

Physical Security: the data centre environment provides superior physical security for the workloads compared to the shop floor environment that traditionally hosted the MES servers. Some of the common approaches are restricting the physical access to the data centre to selected personnel, biometric access control and permanent surveillance.

6.2. Virtualization limitations

Alongside with the benefits above described, there is a set of limitations of the virtualization approach presented for MES:

Real time operations: the main limitation is assuring a virtualized environment can keep up with the real time manufacturing system. Although, initial event propagation tests performed in the pilot implementation show promising results for most applications, this still remains the major limitation. The legacy systems were able to provide guaranteed just-in-time handling to shop floor events, which cannot be completely assured by virtualized environments due to their core nature of shared hardware environments. For some real time manufacturing systems, this still remains the main limitation of adopting virtualization at MES layer. For example, the control of a robot with real time corrections, that needs to be computed on the fly, the network latency does have a severe performance impact on the system, almost doubling the execution time for the specific operation. For this reason, the implementation should consider these scenarios and use physical systems when required to assure acceptable performance.

Development costs: moving from the legacy system to a near real-time virtualized implementation adds a series of development costs to the enterprises. Currently, shop floor device manufacturers provide limited or no support for virtualizing their hardware, so the costs of implementing integrations for bridging the physical devices to private cloud workloads and other required adapters fall within the implementing company area. These costs can become significant for large implementations.

Initial investment: the initial investment in the private cloud architecture for MES virtualization considering both the

hardware purchase and the associated software can represent an obstacle, especially if the private cloud is dedicated to the MES virtualization project. The effectiveness of the private cloud in this context depends on the sharing of the resources among business units in order to achieve optimal resource utilization and to reduce the operational costs.

Training and know how: like with any cutting age technology adoption, virtualization has an associated limitation in finding the knowledgeable resources that can implement it and exploit it to the full potential. Training of the engineers designing and implementing such a solution is an important pre-requisite for a successful implementation.

7. Conclusions and future work

This paper introduces the vMES virtualization as a solution to reduce operational costs and improve the flexibility, agility, reconfigurability and maintainability of the manufacturing system. The MES functions are mapped to specific workloads that are defined in terms of resource requirements and scaling profiles.

Virtualization brings many advantages on the manufacturing system reliability too by allowing full system snapshots and backups and quick recovery in case of failures, as well as providing built-in redundancy. Most private cloud implementations offer these features by default and can be directly adopted. MES workload virtualization allows a separation or decoupling between the physical resources and the controlling information system. The most important advantage introduced by decoupling is the possibility to have multiple versions of the MES workload with different configurations and switch between them as required. This reconfiguring capability is very useful when initial configuration or changes need to be tested and implemented on the production line. With virtualized MES workloads a redundancy mechanism with active/active or active/passive workloads can be implemented. In traditional implementations of manufacturing cells, implementing a MES component redundancy involves the use of a separate physical server, and so it induces a high operational overhead. However, without controller redundancy it is impossible to assure high availability for MES components. A private cloud implementation would be able to detect a workload failure and switch to the redundant workload in a very short time, assuring a high uptime for the entire MES environment.

Adoption of virtualization and cloud related functionality, like automatic provision and de-provision of workloads and predictive resource allocation represent the first steps for implementing highly flexible manufacturing systems designed on SOA concepts and patterns. In the recent years a great deal of effort has been invested in these cloud platforms by both academia and cloud vendors. Manufacturing enterprises have a new opportunity to take advantage of this and move from legacy systems to highly scalable and flexible solutions based on cloud platforms.

Future work is focused on developing an integrated framework for vMES virtualization based on the concepts presented in this paper.

References

- [1] C.D. Weissman, S. Bobrowski, The design of the force.com multitenant internet application development platform, in: *Proceedings of the 35th SIGMOD Int. Conf. on Management of Data*, 2009, pp. 889–896.
- [2] B. Scholten, The road to integration: a guide to applying the ISA-95 standard in manufacturing, ISA, 2007.
- [3] M. Witsch, B. Vogel-Heuser, Towards a formal specification framework for manufacturing execution systems, *IEEE Transactions on Industrial Informatics* 8 (2) (2012) 311–320.

- [4] S. Leimeister, M. Böhm, C. Riedl, H. Krcmar, The business perspective of cloud computing: actors, roles and value networks, in: 18th European Conference on Information Systems, ECIS 2010, 2010.
- [5] J. McCarthy, Comments on time sharing, Communications of the ACM 10 (9) (1967) (ACM, New York, NY, USA).
- [6] P. Mell, G. Timothy, The NIST definition of cloud computing (draft), in: NIST Special Publication 800, no. 145, 2011, p. 7.
- [7] Y.C. Lee, A.Y. Zomaya, Energy efficient utilization of resources in cloud computing systems, Journal of Supercomputing 60 (2) (2012) 268–280.
- [8] A. Gavrilovska, S. Kumar, H. Raj, K. Schwan, V. Gupta, R. Nathuji, P. Saraiya, High-performance hypervisor architectures: virtualization in HPC systems, in: Workshop on System-level Virtualization for HPC, 2007.
- [9] M.F. Mergen, V. Uhlig, O. Krieger, J. Xenidis, Virtualization for high-performance computing, ACM SIGOPS – Operating Systems Review 40 (2) (2006) 8–11.
- [10] H. Raj, K. Schwan, High performance and scalable I/O virtualization via self-virtualized devices, in: Proceedings of the 16th Int. Symposium on High Performance Distributed Computing ACM, 2007, pp. 179–188.
- [11] D. Jamil, H. Zaki, Security issues in cloud computing and countermeasures, International Journal of Engineering Science and Technology 3 (4) (2011) 2672–2676.
- [12] L.M. Kaufman, Data security in the world of cloud computing, IEEE Security & Privacy 7 (4) (2009) 61–64.
- [13] D. Svantesson, R. Clarke, Privacy and consumer risks in cloud computing, Computer Law & Security Review 26 (4) (2010) 391–397.
- [14] S. Wang, L. Zhipiao, S. Qibo, Z. Hua, Y. Fangchun, Towards an accurate evaluation of quality of cloud service in service-oriented cloud computing, Journal of Intelligent Manufacturing (2012) 1–9.
- [15] X. Wang, W.X. Xun, ICMS: a cloud-based manufacturing system, in: Cloud Manufacturing, Springer, London, 2013, pp. 1–22.
- [16] W. Lei, Resource virtualization model in cloud manufacturing, Advanced Materials Research 143 (2011) (2011) 1250–1253.
- [17] Y. Cheng, F. Tao, L.Z.X. Zhang, G.H. Xi, D. Zhao, Study on the utility model and utility equilibrium of resource service transaction in cloud manufacturing, in: IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), 2010, 2298–2302.
- [18] B.H. Li, L. Zhang, S.L. Wang, F. Tao, J.W. Cao, X.D. Jiang, X. Song, X.D. Chai, Cloud manufacturing: a new service-oriented networked manufacturing model, Computer Integrated Manufacturing Systems 16 (1) (2010) 1–7.
- [19] X. Xu, From cloud computing to cloud manufacturing, Robotics and Computer-Integrated Manufacturing 28 (1) (2012) 75–86.
- [20] L. Zhang, L. Yongliang, T. Fei, H.L. Bo, R. Lei, Z. Xuesong, G. Hua, C. Ying, H. Anrui, L. Yongkui, Cloud manufacturing: a new manufacturing paradigm, Enterprise Information Systems (2012) 1–21 (ahead-of-print).
- [21] S. Shi, R. Mo, H. Yang, Z. Chang, Z. Chen, An implementation of modelling resources in a manufacturing grid for resource sharing, International Journal of Computer Integrated Manufacturing 20 (2007) 169–177.
- [22] N. Liu, X. Li, A resource virtualization mechanism for cloud manufacturing systems, in: M. Sinderen, P. Johnson, X. Xu, G. Doumeingts (Eds.), Enterprise Interoperability, Lecture Notes in Business Information Processing, vol. 122, Springer, Berlin, Heidelberg, 2012, pp. 46–59.
- [23] L. Wu, C. Yang, A solution of manufacturing resources sharing in cloud computing environment, in: Cooperative Design Visualization and Engineering, Springer, Berlin, Heidelberg, 2010, pp. 247–252.
- [24] Y. Luo, L. Zhang, F. Tao, L. Ren, Y. Liu, Z. Zhang, A modeling and description method of multidimensional information for manufacturing capability in cloud manufacturing system, International Journal on Advanced Manufacturing Technologies (2013) 1–15.
- [25] P. Leitão, J. Marco, A. Bepperling, D. Cachapa, A.W. Colombo, F. Restivo, Integration of virtual and real environments for engineering service-oriented manufacturing systems, Journal of Intelligent Manufacturing 23 (6) (2012) 2551–2563.
- [26] R. Babiceanu, F. Chen, Development and applications of holonic manufacturing systems: a survey, Journal of Intelligent Manufacturing 17 (2006) 111–131 (Springer).
- [27] Th. Borangiu, A service-oriented architecture for holonic manufacturing control, Springer Series Studies in Computational Intelligence, vol. 243, Springer, 2009, pp. 489–504.
- [28] C. Morariu, O. Morariu, Th. Borangiu, Manufacturing service bus integration model for implementing highly flexible and scalable manufacturing systems, in: Proceedings of the 14th IFAC INCOM'12 Symposium, Papers On Line, Vol. 14, Part 1, (2012), pp. 1850–1855.
- [29] G. Meyer, K. Främling, J. Holmström, Intelligent products: a survey, Computers in Industry 60 (3) (2009) 137–148.
- [30] C. Morariu, Th. Borangiu, Manufacturing integration framework: a SOA perspective on manufacturing, Information Control Problems in Manufacturing 14 (1) (2012) 31–38.
- [31] D.A. Chappell, Enterprise Service Bus, O'Reilly Media Inc., 2009.
- [32] D. Abhijit, D. Wagle, Delivering software as a service, The McKinsey Quarterly, issue 6/2007 (2007).
- [33] C. Morariu, O. Morariu, Th. Borangiu, Customer order management in service oriented holonic manufacturing, Computers in Industry 64 (8) (2013) 1061–1072.

Web references

ebXML Standard, 2013, available online www.ebxml.org. STEP Standard, 2013, available online www.steptools.com. OAG BOD Standard, Open Applications Group Integration Specification (OAGIS), 2013, available online www.oagi.org. VMware ROI TCO Calculator for Server and Desktop Virtualization, 2013, available online <http://roitco.vmware.com/vmw/>.



Octavian Morariu has obtained his M.Sc. degree in Computer Science in 2006 at Technical University of Cluj-Napoca and his Ph.D. degree in Systems Engineering in 2014 at University Politehnica of Bucharest, Faculty of Automatic Control and Computers. His research work is focused on Private Cloud Computing systems and their optimization for the use in manufacturing systems integration. Octavian was awarded for his research the IBM Ph.D. Fellowship award for 2012–2013.



Theodor Borangiu obtained the M.Sc. degree in Automatic Control in 1971 and the Ph.D. degree in 'Control Systems' in 1979 of the University Politehnica of Bucharest. He is professor since 1991 in the Dept. of Automatics and Applied Informatics, Head of the Robotics & AI Laboratory of the University Politehnica of Bucharest. Director of Ph.D. thesis since 1992 in 'Control Systems'. Member of the Senate of University Politehnica of Bucharest (2004–present). Visiting professor at the universities of Vienna (1992–1994), Krems (1994), Cartagena (1996–1998), Toluca (1996), Darmstadt (1990), Konstanz (1992, 2002–2003), Valenciennes (1996–2009). He has published 38 monographs, 238 scientific papers and is author of 5 patents, all in the

field of advanced robot control, machine vision, multiprocessor control, CIM, FMS, PLC systems, and AI applied in holonic, multi-agent manufacturing control and robotics.



Silviu Raileanu is currently a Ph.D. student and teaching assistant at the University Politehnica Bucharest, Faculty of Automatic Control and Computers. He received both his engineering degree (2005) and his masters degree (2009) in automatics from the Faculty of Automatic Control and Computers. He holds laboratories and seminars for Robot Control Systems and Design of Digital Devices. His interests include distributed and heterarchical control of discrete event systems, scheduling and planning of flexible fabrication systems, the application of multi-agent and holonic theories for the control of flexible manufacturing systems, domains in which he published and contributed to several publications.